ECE 2011
Introduction to Computer Engineering
Spring 2026

Lecture 11
02-17-2026

Linghao Song
Yale ECE

Always Statement

General Structure:

```
always @ (sensitivity list)
   statement;
```

Whenever the event in the `sensitivity list` occurs, the `statement` is executed

Copyright © 2007 Elsevier

Combinational Logic using **always**

```
// combinational logic using an always statement
module gates(input       [3:0] a, b,
             output reg [3:0] y1, y2, y3, y4, y5);
  always @(*)        // need begin/end because there is
    begin             // more than one statement in always
      y1 = a & b;     // AND
      y2 = a | b;     // OR
      y3 = a ^ b;     // XOR
      y4 = ~(a & b); // NAND
      y5 = ~(a | b); // NOR
    end
endmodule
```

This hardware could be described with assign statements using fewer lines of code, so it's better to use assign statements in this case.

Copyright © 2007 Elsevier

Combinational Logic using case

```
module sevenseg(input       [3:0] data,
                output reg [6:0] segments);
  always @(*)
    case (data)
      //                abc_defg
      0: segments = 7'b111_1110;
      1: segments = 7'b011_0000;
      2: segments = 7'b110_1101;
      3: segments = 7'b111_1001;
      4: segments = 7'b011_0011;
      5: segments = 7'b101_1011;
      6: segments = 7'b101_1111;
      7: segments = 7'b111_0000;
      8: segments = 7'b111_1111;
      9: segments = 7'b111_1011;
      default: segments = 7'b000_0000; // required
    endcase
endmodule
```

Copyright © 2007 Elsevier

Combinational Logic using `case`

- In order for a `case` statement to imply combinational logic, all possible input combinations must be described by the HDL.
- Remember to use a **`default`** statement when necessary.

Copyright © 2007 Elsevier

Rules for Signal Assignment

c'est quoi ?

- Use `always @(posedge clk)` and nonblocking assignments (<=) to model synchronous sequential logic

```
always @ (posedge clk)
  q <= d; // nonblocking
```

- Use continuous assignments (assign …)to model simple combinational logic.

```
assign y = a & b;
```

- Use `always @ (*)` and blocking assignments (=) to model more complicated combinational logic where the `always` statement is helpful.

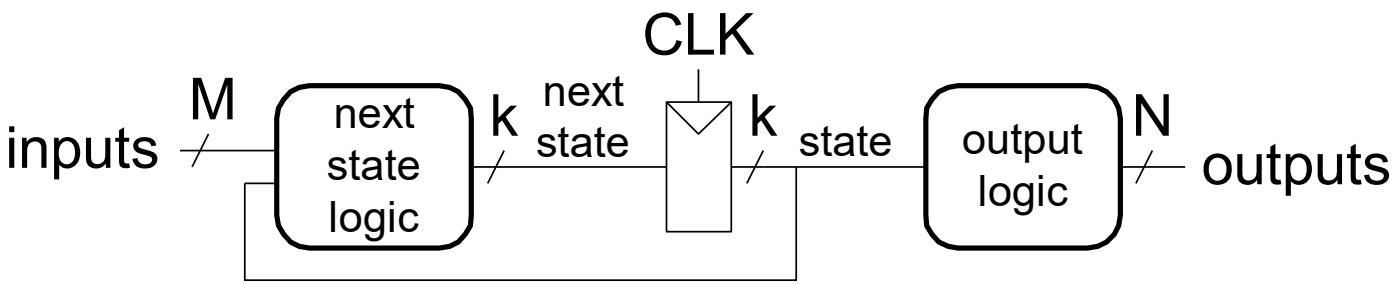
- Do not make assignments to the same signal in more than one `always` statement or continuous assignment statement.

Copyright © 2007 Elsevier

ELSEVIER

Finite State Machines (FSMs)

- Three blocks:
  - next state logic
  - state register
  - output logic



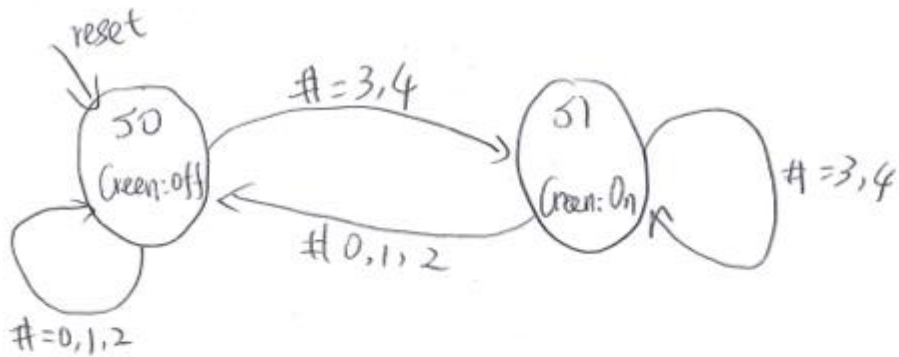
Copyright © 2007 Elsevier

FSM in Verilog

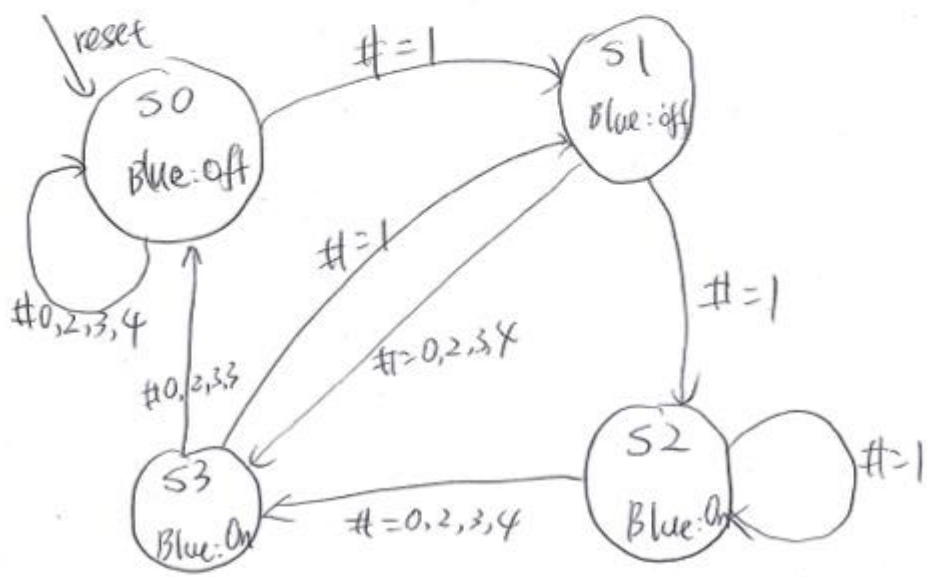
```
module divideby3FSM (input  clk,
                     input  reset,
                     output q);
   reg  [1:0] state, nextstate;

   parameter S0 = 2'b00;
   parameter S1 = 2'b01;
   parameter S2 = 2'b10;

// state register
   always @ (posedge clk, posedge reset)
      if (reset) state <= S0;
      else       state <= nextstate;
 // next state logic
   always @ (*)
      case (state)
         S0:      nextstate = S1;
         S1:      nextstate = S2;
         S2:      nextstate = S0;
         default: nextstate = S0;
      endcase
   // output logic
   assign q = (state == S0);
endmodule
```

Copyright © 2007 Elsevier

Green Light FSM



Blue Light FSM



Green FSM

current state	input NYBS	next state
S0 /00	0,1,2	S0 /0
S0 /00	3,4	S1 /1
S1 /1	0,1,2	S0 /0
S1 /1	3,4	S1 /1

~~0,1,2 = $\bar{N}\bar{Y}\bar{B}\bar{S}$ +~~ C 3,4 = ~~$\bar{Y}\bar{B}\bar{S}$ +~~ $N\bar{Y}B + \bar{Y}BS + N\bar{Y}S + NBS$

$\bar{C}$ = 0,1,2 = $\overline{3,4}$

Blue

current state	input	next state
S0 /00	0,2,3,4	S0 /00
S0 /00	1	S1 /01
S1 /01	0,2,3,4	S3 /11
S1 /01	1	S2 /10
S2 /10	0,2,3,4	S3 /11
S2 /10	1	S2 /10
S3 /11	0,2,3,4	S0 /00
S3 /11	1	S1 /01

C, #1 = $N\bar{Y}\bar{B}\bar{S} + \bar{N}\bar{Y}\bar{B}\bar{S} + \bar{N}\bar{Y}B\bar{S} + \bar{N}\bar{Y}\bar{B}S$

$\bar{C}$, #0,2,3,4

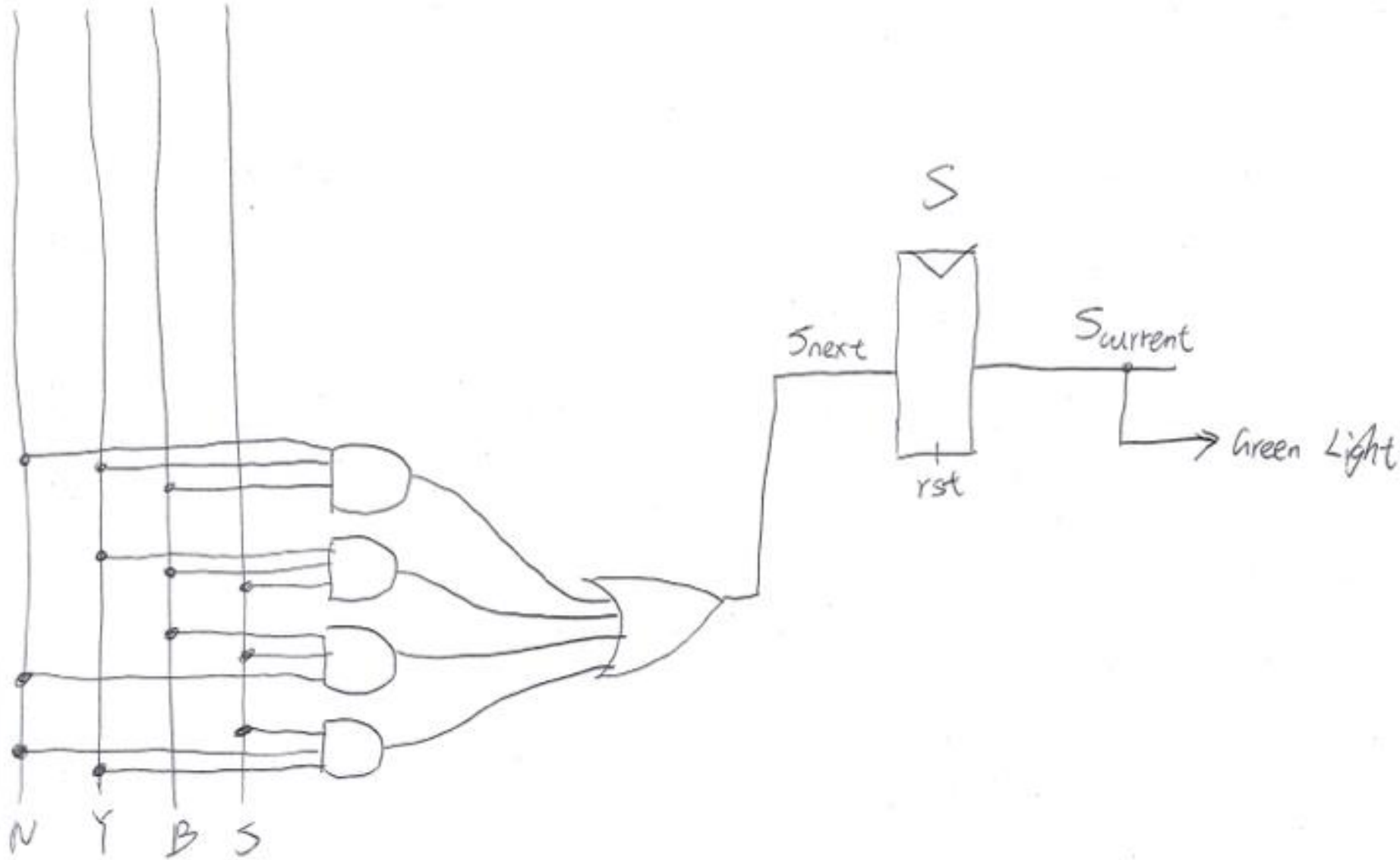
Green FSM / output table

Current State	Output
S0 /0	Green : off
S1 /1	Green : On

Blue FSM

current state	output
S0	Blue : off
S1	Blue : off
S2	Blue : On
S3	Blue : On

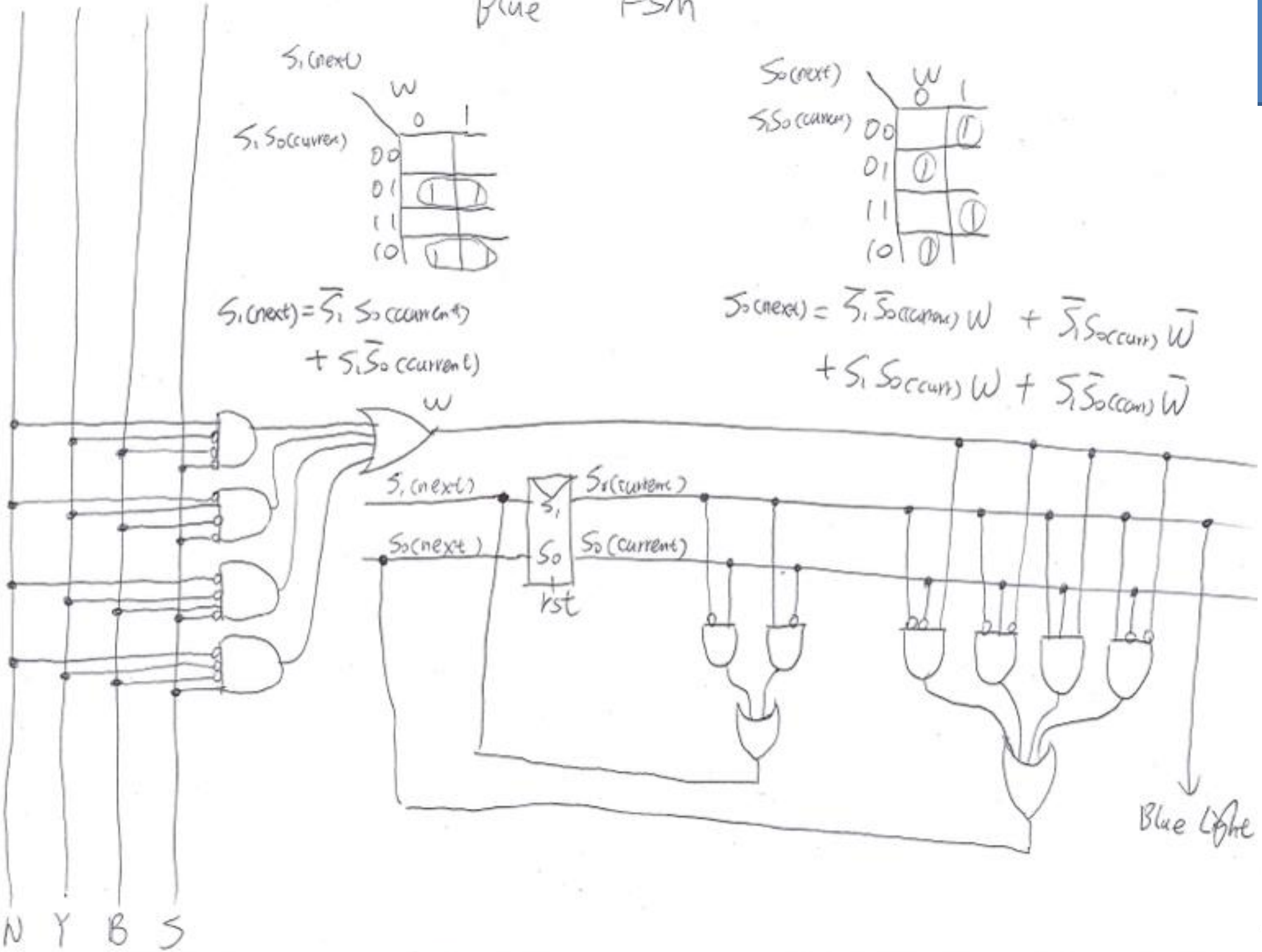
Green FSM



Blue FSM

$$S_1(next) = \bar{S}_1 S_0(current) + S_1 \bar{S}_0(current)$$

$$S_0(next) = \bar{S}_1 \bar{S}_0(current) W + \bar{S}_1 S_0(curr) \bar{W} + S_1 S_0(curr) W + S_1 \bar{S}_0(curr) \bar{W}$$



Testbenches

- HDL code written to test another HDL module, the *device under test* (dut), also called the *unit under test* (uut)
- Not synthesizeable
- Types of testbenches:
  - Simple testbench
  - Self-checking testbench
  - Self-checking testbench with testvectors

Copyright © 2007 Elsevier

ELSEVIER

Simple Testbench

```
module testbench1();
  reg  a, b, c;
  wire y;
  // instantiate device under test
  sillyfunction dut(a, b, c, y);
  // apply inputs one at a time
  initial begin
    a = 0; b = 0; c = 0; #10;
    c = 1; #10;
    b = 1; c = 0; #10;
    c = 1; #10;
    a = 1; b = 0; c = 0; #10;
    c = 1; #10;
    b = 1; c = 0; #10;
    c = 1; #10;
  end
endmodule
```

10 ns.

→ generate values of a,b,c & input into dut

Copyright © 2007 Elsevier

Self-checking Testbench

```
module testbench2();
  reg  a, b, c;
  wire y;
  // instantiate device under test
  sillyfunction dut(a, b, c, y);
  // apply inputs one at a time
  // checking results
  initial begin
    a = 0; b = 0; c = 0; #10;
    if (y !== 1) $display("000 failed.");
    c = 1; #10;
    if (y !== 0) $display("001 failed.");
    b = 1; c = 0; #10;
    if (y !== 0) $display("010 failed.");
    c = 1; #10;
    if (y !== 0) $display("011 failed.");
    a = 1; b = 0; c = 0; #10;
    if (y !== 1) $display("100 failed.");
    c = 1; #10;
    if (y !== 1) $display("101 failed.");
    b = 1; c = 0; #10;
    if (y !== 0) $display("110 failed.");
    c = 1; #10;
    if (y !== 0) $display("111 failed.");
  end
endmodule
```

that's very cool!

Copyright © 2007 Elsevier

Testbenches

- HDL code written to test another HDL module, the *device under test* (dut), also called the *unit under test* (uut)
- Not synthesizeable
- Types of testbenches:
  - Simple testbench
  - Self-checking testbench
  - Self-checking testbench with testvectors
- **Read textbook to learn how to write testbenches!**

Copyright © 2007 Elsevier

ELSEVIER

Digital Building Blocks

this is very cool!

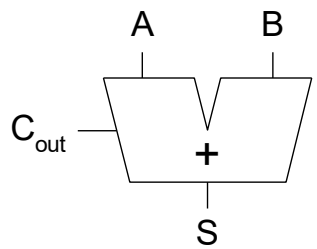
- **Digital building blocks:**
  - Gates, multiplexers, decoders, registers, arithmetic circuits, counters, memory arrays, logic arrays
- **Building blocks demonstrate hierarchy, modularity, and regularity:**
  - Hierarchy of simpler components
  - Well-defined interfaces and functions
  - Regular structure easily extends to different sizes
- **We'll use these building blocks to build a microprocessor**

© *Digital Design and Computer Architecture*, 2nd Edition, 2012

ELSEVIER

1-Bit Adders

Half Adder

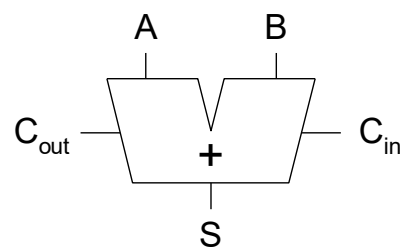


A	B	C_{out}	S
0	0		
0	1		
1	0		
1	1		

S =

C_{out} =

Full Adder



C_{in}	A	B	C_{out}	S
0	0	0		
0	0	1		
0	1	0		
0	1	1		
1	0	0		
1	0	1		
1	1	0		
1	1	1		

S =

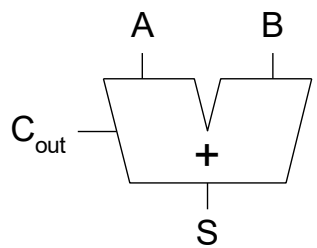
C_{out} =

© *Digital Design and Computer Architecture*, 2nd Edition, 2012

ELSEVIER

1-Bit Adders

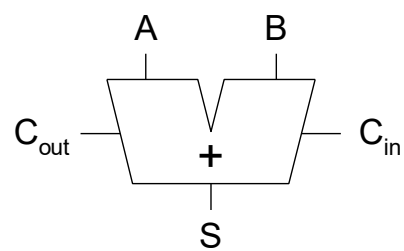
Half Adder



A	B	C_{out}	S
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

S =
C_{out} =

Full Adder



C_{in}	A	B	C_{out}	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

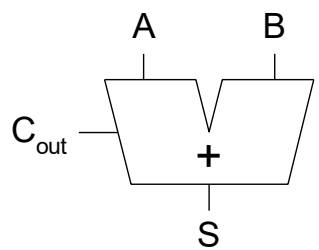
S =
C_{out} =

© *Digital Design and Computer Architecture*, 2nd Edition, 2012

ELSEVIER

1-Bit Adders

Half Adder

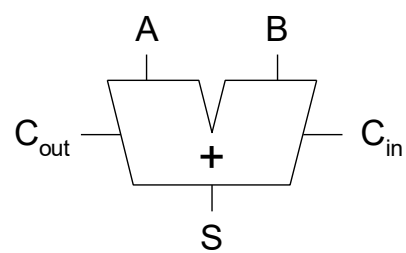


A	B	C_{out}	S
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

$S = A \oplus B$

$C_{out} = AB$

Full Adder



C_{in}	A	B	C_{out}	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

$S = A \oplus B \oplus C_{in}$

$C_{out} = AB + AC_{in} + BC_{in}$

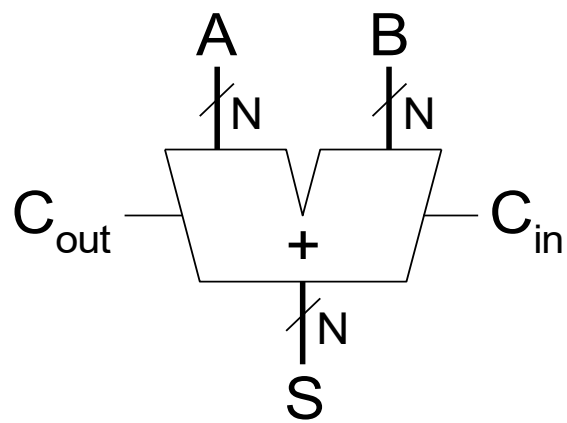
© *Digital Design and Computer Architecture*, 2nd Edition, 2012

ELSEVIER

Multibit Adders (CPAs)

- Types of carry propagate adders (CPAs):
  - **Ripple-carry** (slow)
  - **Carry-lookahead** (fast)
  - **Prefix** (faster)
- Carry-lookahead and prefix adders faster for large adders but require more hardware

Symbol

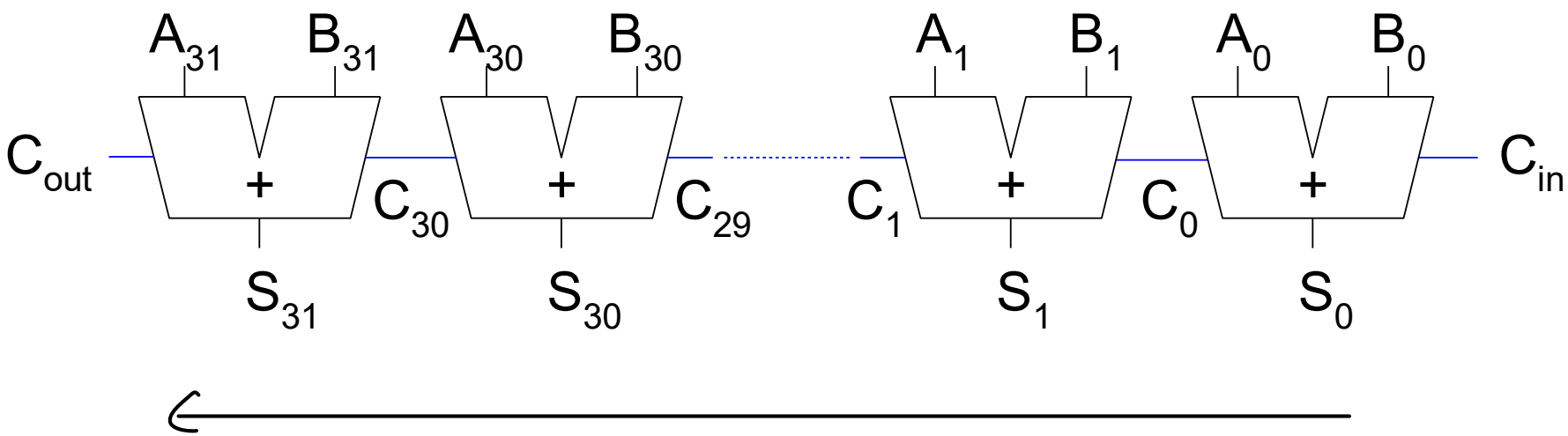


© *Digital Design and Computer Architecture*, 2nd Edition, 2012

ELSEVIER

Ripple-Carry Adder

- Chain 1-bit adders together
- Carry ripples through entire chain
- Disadvantage: **slow**



© *Digital Design and Computer Architecture*, 2nd Edition, 2012

ELSEVIER

Ripple-Carry Adder Delay

$$t_{\text{ripple}} = Nt_{FA}$$

where t_{FA} is the delay of a 1-bit full adder

© *Digital Design and Computer Architecture*, 2nd Edition, 2012

ELSEVIER

Carry-Lookahead Adder

- Compute carry out (C_{out}) for k-bit blocks using *generate* and *propagate* signals
- **Some definitions:**
  - Column i produces a carry out by either ***generating*** a carry out or ***propagating*** a carry in to the carry out
  - Generate (G_i) and propagate (P_i) signals for each column:
    - **Generate:** Column i will generate a carry out if A_i AND B_i are both 1.

$$G_i = A_i B_i$$

    - **Propagate:** Column i will propagate a carry in to the carry out if A_i OR B_i is 1.

$$P_i = A_i + B_i$$

    - **Carry out:** The carry out of column i (C_i) is:

$$C_i = A_i B_i + (A_i + B_i) C_{i-1} = G_i + P_i C_{i-1}$$

© *Digital Design and Computer Architecture*, 2nd Edition, 2012

ELSEVIER

Propagate and Generate Signals

Examples: Column propagate and generate signals:

Column propagate: $P_i = A_i + B_i$

Column generate: $G_i = A_i B_i$

1011	$A_{3:0}$
+ 0110	$B_{3:0}$
1111	P_3, P_2, P_1, P_0
0010	G_3, G_2, G_1, G_0

1011	$A_{3:0}$
+ 1001	$B_{3:0}$
1011	P_3, P_2, P_1, P_0
1001	G_3, G_2, G_1, G_0

© *Digital Design and Computer Architecture*, 2nd Edition, 2012

ELSEVIER

Propagate and Generate Signals

Examples: Column propagate and generate signals:

Column propagate: $P_i = A_i + B_i$

Column generate: $G_i = A_i B_i$

1011	$A_{3:0}$
+ 0110	$B_{3:0}$
1111	P_3, P_2, P_1, P_0
0010	G_3, G_2, G_1, G_0

1011	$A_{3:0}$
+ 1001	$B_{3:0}$
1011	P_3, P_2, P_1, P_0
1001	G_3, G_2, G_1, G_0

$$C_i = G_i + P_i C_{i-1}$$

© *Digital Design and Computer Architecture*, 2nd Edition, 2012

ELSEVIER

Propagate and Generate Signals

Examples: Column propagate and generate signals:

Column propagate: $P_i = A_i + B_i$

Column generate: $G_i = A_i B_i$

Carry-out (C_3) Carry-in (C_{-1})

11111	$C_3, C_2, C_1, C_0, C_{-1}$
1011	$A_{3:0}$
+ 0110	$B_{3:0}$
10010	
1111	P_3, P_2, P_1, P_0
0010	G_3, G_2, G_1, G_0

Carry-out (C_3) Carry-in (C_{-1})

10110	$C_3, C_2, C_1, C_0, C_{-1}$
1011	$A_{3:0}$
+ 1001	$B_{3:0}$
0	
1011	P_3, P_2, P_1, P_0
1001	G_3, G_2, G_1, G_0

$C_3 = G_3 + P_3 C_2$

$C_i = G_i + P_i C_{i-1}$

$1 = 0 + 1.$

© *Digital Design and Computer Architecture*, 2nd Edition, 2012

ELSEVIER

Block Propagate & Generate Signals

Now use column Propagate and Generate signals to compute ***Block Propagate*** and ***Block Generate*** signals for k-bit blocks, i.e.:

- Compute if a k-bit group will **propagate** a carry in (of the block) to the carry out (of the block)
- Compute if a k-bit group will **generate** a carry out (of the block)

ELSEVIER

Block Propagate & Generate Signals

- **Example:** 4-bit blocks
  - Block propagate signal: $\boldsymbol{P_{3:0}}$ (single-bit signal)
  - Block generate signal: $\boldsymbol{G_{3:0}}$ (single-bit signal)

© *Digital Design and Computer Architecture*, 2nd Edition, 2012

ELSEVIER

Block Propagate Signal

- **Example:** 4-bit blocks
  - Block propagate signal: $\boldsymbol{P_{3:0}}$ (single-bit signal)
    - A carry-in would propagate through all 4 bits of the block:

$$P_{3:0} = P_3 P_2 P_1 P_0$$

© *Digital Design and Computer Architecture*, 2nd Edition, 2012

ELSEVIER

Block Propagate Signal Examples

- **Example:** 4-bit blocks
  - Block propagate signal: $P_{3:0}$ (single-bit signal)
    - A carry-in would propagate through all 4 bits of the block:

$$P_{3:0} = P_3 P_2 P_1 P_0$$

    - **Examples:**

1011	$A_{3:0}$
+ 0100	$B_{3:0}$
1111	P_3, P_2, P_1, P_0
0000	G_3, G_2, G_1, G_0

$$\boxed{P_{3:0} = P_3 P_2 P_1 P_0 = 1}$$

1011	$A_{3:0}$
+ 0001	$B_{3:0}$
1011	P_3, P_2, P_1, P_0
0001	G_3, G_2, G_1, G_0

$$\boxed{P_{3:0} = P_3 P_2 P_1 P_0 = 0}$$

© *Digital Design and Computer Architecture*, 2nd Edition, 2012

ELSEVIER

Block Propagate Signal Examples

- **Example:** 4-bit blocks
  - Block propagate signal: $P_{3:0}$ (single-bit signal)
    - A carry-in would propagate through all 4 bits of the block:

$$P_{3:0} = P_3 P_2 P_1 P_0$$

    - **Examples:**

Carry-out (C_3) Carry-in (C_{-1})

11111	$C_3, C_2, C_1, C_0, C_{-1}$
1011	$A_{3:0}$
+ 0100	$B_{3:0}$
0000	$S_{3:0}$
1111	P_3, P_2, P_1, P_0
0000	G_3, G_2, G_1, G_0

$P_{3:0} = P_3 P_2 P_1 P_0 = 1$

Carry-out (C_3) Carry-in (C_{-1})

00111	$C_3, C_2, C_1, C_0, C_{-1}$
1011	$A_{3:0}$
+ 0001	$B_{3:0}$
1101	$S_{3:0}$
1011	P_3, P_2, P_1, P_0
0001	G_3, G_2, G_1, G_0

$P_{3:0} = P_3 P_2 P_1 P_0 = 0$

© Digital Design and Computer Architecture, 2nd Edition, 2012

ELSEVIER

Block Propagate & Generate Signals

- **Example:** 4-bit blocks
  - Block propagate signal: $\boldsymbol{P_{3:0}}$ (single-bit signal)
    - A carry-in would propagate through all 4 bits of the block:

$$P_{3:0} = P_3 P_2 P_1 P_0$$

  - Block generate signal: $\boldsymbol{G_{3:0}}$ (single-bit signal)

© *Digital Design and Computer Architecture*, 2nd Edition, 2012

ELSEVIER

Block Propagate & Generate Signals

- **Example:** 4-bit blocks
  - Block propagate signal: $P_{3:0}$ (single-bit signal)
    - A carry-in would propagate through all 4 bits of the block:

$$P_{3:0} = P_3 P_2 P_1 P_0$$

  - Block generate signal: $G_{3:0}$ (single-bit signal)
    - A carry is generated:
      - in column 3, **or**
      - in column 2 and propagated through column 3, **or**
      - in column 1 and propagated through columns 2 and 3, **or**
      - in column 0 and propagated through columns 1-3

© *Digital Design and Computer Architecture*, 2nd Edition, 2012

ELSEVIER

Block Propagate & Generate Signals

- **Example:** 4-bit blocks
  - Block propagate signal: $P_{3:0}$ (single-bit signal)
    - A carry-in would propagate through all 4 bits of the block:

$$P_{3:0} = P_3 P_2 P_1 P_0$$

  - Block generate signal: $G_{3:0}$ (single-bit signal)
    - A carry is generated:
      - in column 3, **or**
      - in column 2 and propagated through column 3, **or**
      - in column 1 and propagated through columns 2 and 3, **or**
      - in column 0 and propagated through columns 1-3

$$G_{3:0} = G_3 + G_2 P_3 + G_1 P_2 P_3 + G_0 P_1 P_2 P_3$$

© *Digital Design and Computer Architecture*, 2nd Edition, 2012

ELSEVIER

Block Generate Signal Examples

- **Example:** 4-bit blocks
  - Block generate signal: $G_{3:0}$ (single-bit signal)
    - A carry is: generated in column 3, **or** generated in column 2 and propagated through column 3, **or** ...

$$G_{3:0} = G_3 + G_2P_3 + G_1P_2P_3 + G_0P_1P_2P_3$$

    - **Examples:**

1001	$A_{3:0}$	1110	$A_{3:0}$	0110	$A_{3:0}$
+ 1100	$B_{3:0}$	+ 0100	$B_{3:0}$	+ 0010	$B_{3:0}$
1101	P_3,P_2,P_1,P_0	1110	P_3,P_2,P_1,P_0	0110	P_3,P_2,P_1,P_0
1000	G_3,G_2,G_1,G_0	0100	G_3,G_2,G_1,G_0	0010	G_3,G_2,G_1,G_0
$G_{3:0} = 1$		$G_{3:0} = 1$		$G_{3:0} = 0$	

© *Digital Design and Computer Architecture*, 2nd Edition, 2012

ELSEVIER

Block Generate Signal Examples

- **Example:** 4-bit blocks
  - Block generate signal: $G_{3:0}$ (single-bit signal)
    - A carry is: generated in column 3, **or** generated in column 2 and propagated through column 3, **or** ...

$$G_{3:0} = G_3 + G_2P_3 + G_1P_2P_3 + G_0P_1P_2P_3$$

    - **Examples:**

Carry-out (C_3) ↓		Carry-out (C_3) ↓		Carry-out (C_3) ↓	
10011	C_3,C_2,C_1,C_0,C_{-1}	11000	C_3,C_2,C_1,C_0,C_{-1}	01101	C_3,C_2,C_1,C_0,C_{-1}
1001	$A_{3:0}$	1110	$A_{3:0}$	0110	$A_{3:0}$
+ 1100	$B_{3:0}$	+ 0100	$B_{3:0}$	+ 0010	$B_{3:0}$
0110	$S_{3:0}$	0010	$S_{3:0}$	1000	$S_{3:0}$
1101	P_3,P_2,P_1,P_0	1110	P_3,P_2,P_1,P_0	0110	P_3,P_2,P_1,P_0
1000	G_3,G_2,G_1,G_0	0100	G_3,G_2,G_1,G_0	0010	G_3,G_2,G_1,G_0
$G_{3:0} = 1$		$G_{3:0} = 1$		$G_{3:0} = 0$	

© *Digital Design and Computer Architecture*, 2nd Edition, 2012

ELSEVIER

Carry-Lookahead Adder

- **Example:** 4-bit blocks
  - Block propagate signal: $\boldsymbol{P_{3:0}}$ (single-bit signal)
    - A carry-in would propagate through all 4 bits of the block:

$$P_{3:0} = P_3 P_2 P_1 P_0$$

  - Block generate signal: $\boldsymbol{G_{3:0}}$ (single-bit signal)
    - A carry is generated:
      - in column 3, **or**
      - in column 2 and propagated through column 3, **or** …

$$G_{3:0} = G_3 + G_2 P_3 + G_1 P_2 P_3 + G_0 P_1 P_2 P_3$$

$$G_{3:0} = G_3 + P_3 [G_2 + P_2 (G_1 + P_1 G_0)]$$

$$C_3 = G_{3:0} + P_{3:0} C_{-1}$$

© *Digital Design and Computer Architecture*, 2nd Edition, 2012

ELSEVIER

Carry-Lookahead Adder

- **Example:** 4-bit blocks ($G_{3:0}$ and $P_{3:0}$) :

$$G_{3:0} = G_3 + P_3\,[G_2 + P_2\,(G_1 + P_1 G_0)]$$

$$P_{3:0} = P_3 P_2 P_1 P_0$$

$$C_3 = G_{3:0} + P_{3:0}\,C_{-1}$$

- **Generally,**

$$G_{i:j} = G_i + P_i\,[G_{i-1} + P_{i-1}\,(G_{i-2} + P_{i-2} G_j)]$$

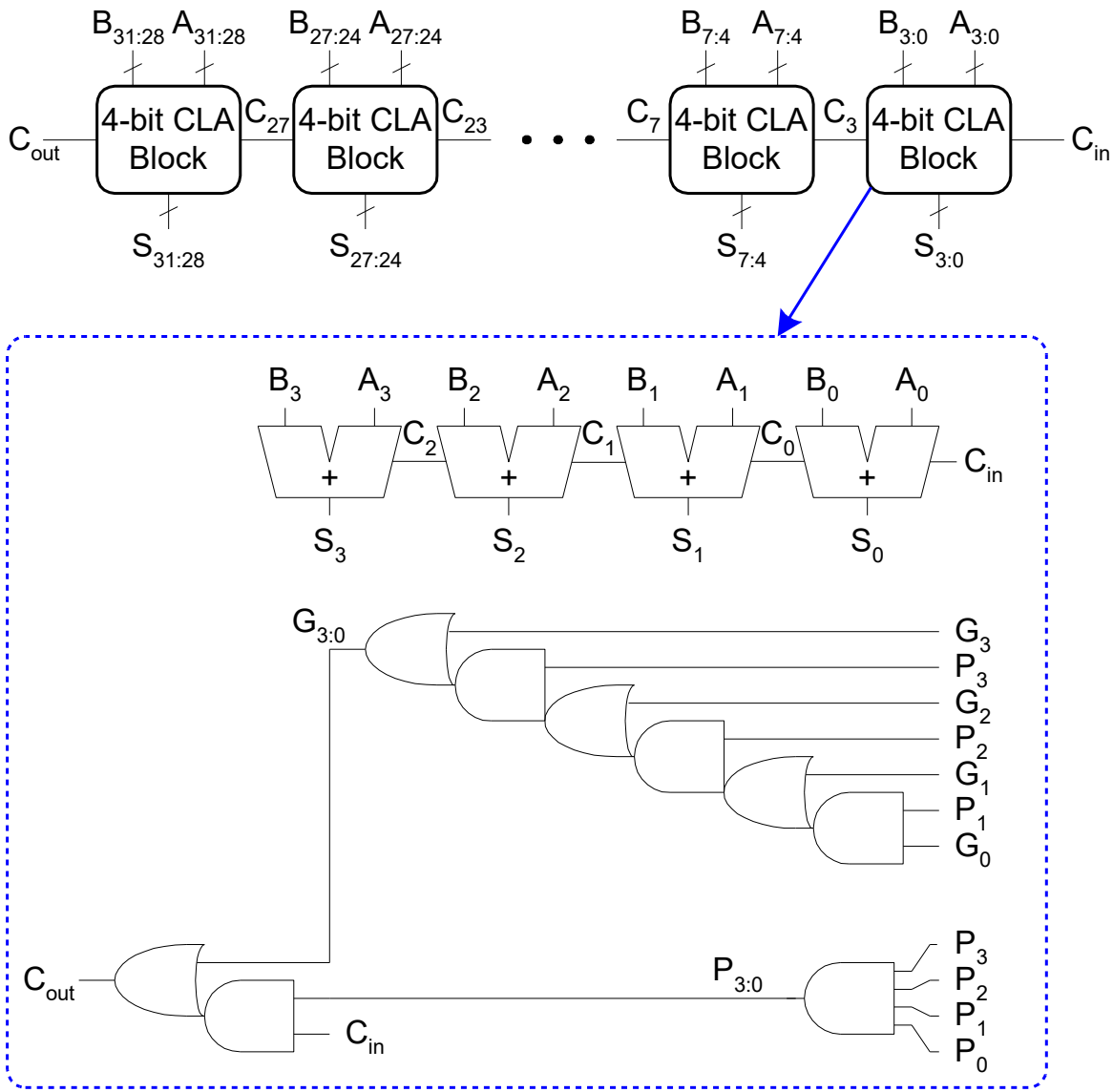
$$P_{i:j} = P_i P_{i-1} P_{i-2} P_j$$

$$C_i = G_{i:j} + P_{i:j}\,C_{j-1}$$

© *Digital Design and Computer Architecture*, 2nd Edition, 2012

ELSEVIER

32-bit CLA with 4-bit Blocks



© *Digital Design and Computer Architecture*, 2nd Edition, 2012

ELSEVIER

Carry-Lookahead Addition

- **Step 1:** Compute G_i and P_i for all columns
- **Step 2:** Compute G and P for k-bit blocks
- **Step 3:** C_{in} propagates through each k-bit propagate/generate logic (meanwhile computing sums)
- **Step 4:** Compute sum for most significant k-bit block

© *Digital Design and Computer Architecture*, 2nd Edition, 2012

ELSEVIER

Carry-Lookahead Addition

- **Step 1:** Compute G_i and P_i for all columns

$$G_i = A_i B_i$$

$$P_i = A_i + B_i$$

© *Digital Design and Computer Architecture*, 2nd Edition, 2012

ELSEVIER

Carry-Lookahead Addition

- **Step 1:** Compute G_i and P_i for all columns
- **Step 2:** Compute G and P for k-bit blocks

$$P_{3:0} = P_3 P_2 P_1 P_0$$

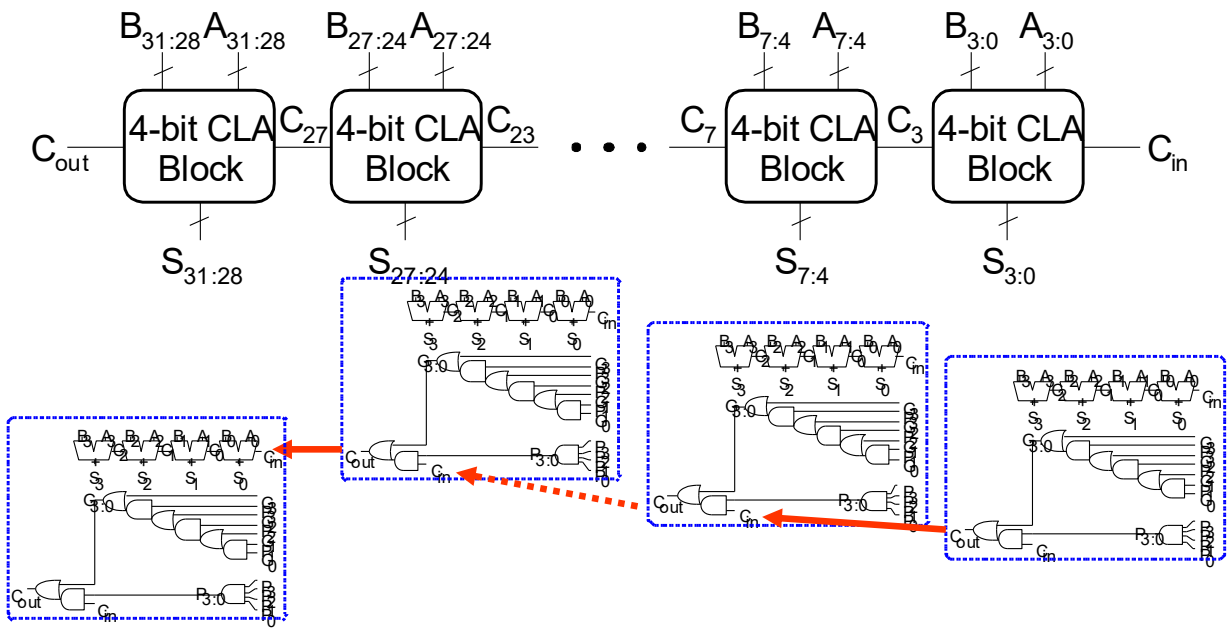
$$G_{3:0} = G_3 + P_3 (G_2 + P_2 (G_1 + P_1 G_0))$$

© *Digital Design and Computer Architecture*, 2nd Edition, 2012

ELSEVIER

Carry-Lookahead Addition

- **Step 1:** Compute G_i and P_i for all columns
- **Step 2:** Compute G and P for k-bit blocks
- **Step 3:** C_{in} propagates through each k-bit propagate/generate logic (meanwhile computing sums)

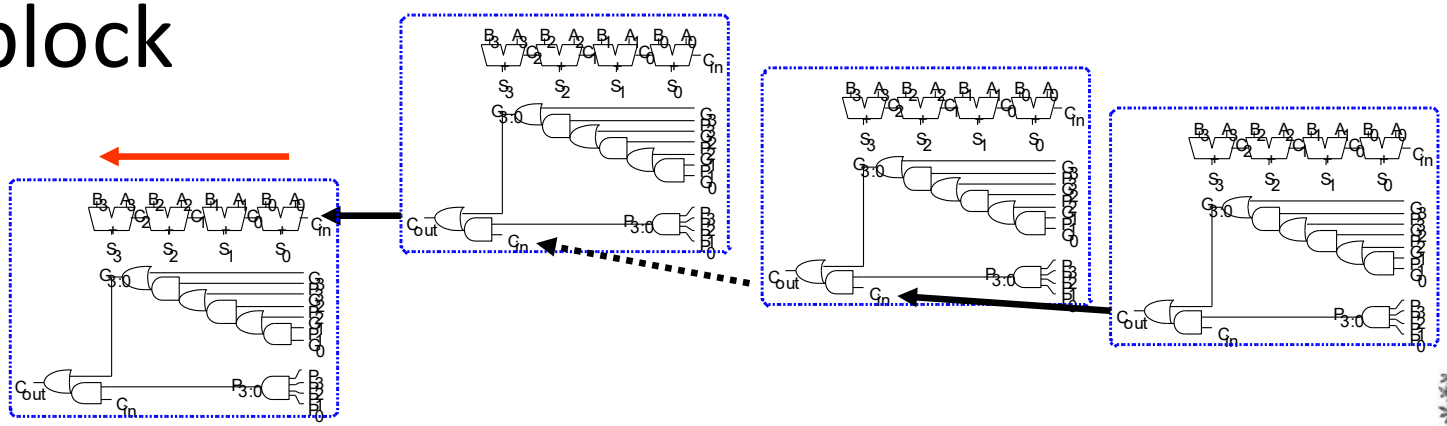


© *Digital Design and Computer Architecture*, 2nd Edition, 2012

ELSEVIER

Carry-Lookahead Addition

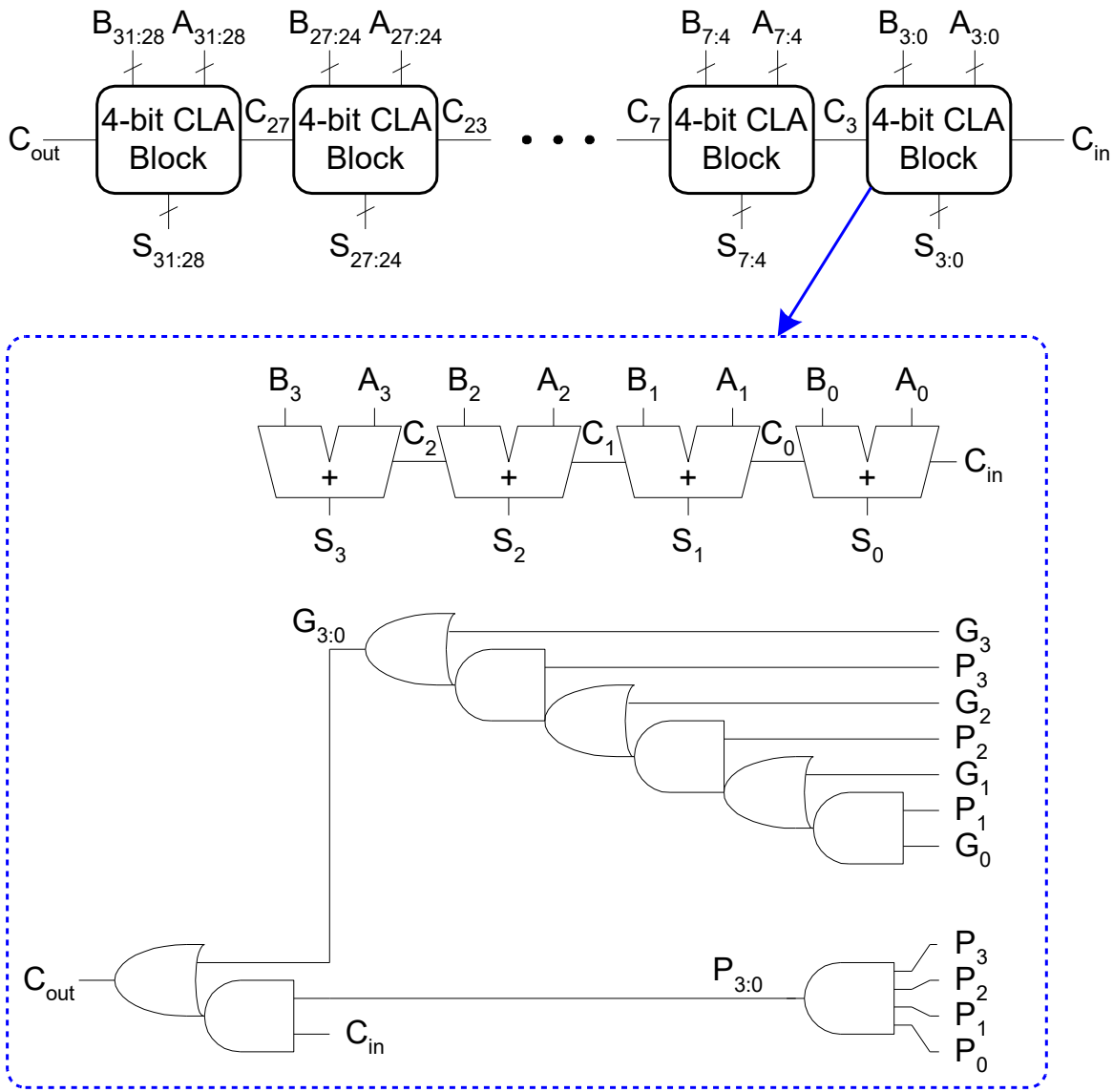
- **Step 1:** Compute G_i and P_i for all columns
- **Step 2:** Compute G and P for k-bit blocks
- **Step 3:** C_{in} propagates through each k-bit propagate/generate logic (meanwhile computing sums)
- **Step 4:** Compute sum for most significant k-bit block



© *Digital Design and Computer Architecture*, 2nd Edition, 2012

ELSEVIER

32-bit CLA with 4-bit Blocks



© *Digital Design and Computer Architecture*, 2nd Edition, 2012

ELSEVIER

Carry-Lookahead Adder Delay

For N-bit CLA with k-bit blocks:

$$t_{CLA} = t_{pg} + t_{pg_block} + (N/k - 1)t_{AND_OR} + kt_{FA}$$

- t_{pg} : delay to generate all P_i, G_i
- t_{pg_block} : delay to generate all $P_{i:j}$, $G_{i:j}$
- t_{AND_OR} : delay from C_{in} to C_{out} of final AND/OR gate in k-bit CLA block

An N-bit carry-lookahead adder is generally much faster than a ripple-carry adder for $N > 16$

© *Digital Design and Computer Architecture*, 2nd Edition, 2012

ELSEVIER